



PDF Download  
3699598.pdf  
24 December 2025  
Total Citations: 10  
Total Downloads:  
1894

Latest updates: <https://dl.acm.org/doi/10.1145/3699598>

RESEARCH-ARTICLE

## Exploring Automated Assertion Generation via Large Language Models

QUANJUN ZHANG, Nanjing University, Nanjing, Jiangsu, China

WEIFENG SUN, Chongqing University, Chongqing, China

CHUNRONG FANG, Nanjing University, Nanjing, Jiangsu, China

BOWEN YU, Nanjing University, Nanjing, Jiangsu, China

HONGYAN LI, Chongqing University, Chongqing, China

MENG YAN, Chongqing University, Chongqing, China

[View all](#)

Open Access Support provided by:

[Chongqing University](#)

[Nanjing University](#)

[Huawei Technologies Co., Ltd.](#)

Published: 23 February 2025  
Online AM: 07 October 2024  
Accepted: 26 September 2024  
Revised: 02 July 2024  
Received: 16 December 2023

[Citation in BibTeX format](#)

# Exploring Automated Assertion Generation via Large Language Models

**QUANJUN ZHANG**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

**WEIFENG SUN**, School of Big Data and Software Engineering, Chongqing University, Chongqing, China

**CHUNRONG FANG** and **BOWEN YU**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

**HONGYAN LI** and **MENG YAN**, School of Big Data and Software Engineering, Chongqing University, Chongqing, China

**JIANYI ZHOU**, Huawei Cloud Computing Technologies Co., Ltd., Beijing, China

**ZHENYU CHEN**, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China

---

Unit testing aims to validate the correctness of software system units and has become an essential practice in software development and maintenance. However, it is incredibly time-consuming and labor-intensive for testing experts to write unit test cases manually, including test inputs (i.e., prefixes) and test oracles (i.e., assertions). Very recently, some techniques have been proposed to apply Large Language Models (LLMs) to generate unit assertions and have proven the potential in reducing manual testing efforts. However, there has been no systematic comparison of the effectiveness of these LLMs, and their pros and cons remain unexplored.

To bridge this gap, we perform the first extensive study on applying various LLMs to automated assertion generation. The experimental results on two independent datasets show that studied LLMs outperform six state-of-the-art techniques with a prediction accuracy of 51.82%–58.71% and 38.72%–48.19%. The improvements achieve 29.60% and 12.47% on average. Besides, as a representative LLM, CodeT5 consistently outperforms all studied LLMs and all baselines on both datasets, with an average improvement of 13.85% and 26.64%,

---

Quanjun Zhang and Weifeng Sun contributed equally to this research.

This work was supported partially by the National Natural Science Foundation of China (61932012, 62372228), CCF-Huawei Populus Grove Fund (CCF-HuaweiSE202304, CCF-HuaweiSY202306), Science, Technology and Innovation Commission of Shenzhen Municipality (CJGJZD20200617103001003), and Open Fund Projects of State Key Laboratory of Intelligent Vehicle Safety Technology (No. IVSTSKL-202412).

Authors' Contact Information: Quanjun Zhang, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: quanjun.zhang@smail.nju.edu.cn; Weifeng Sun, Big Data and Software Engineering, Chongqing University, Chongqing, China; e-mail: weifeng.sun@cqu.edu.cn; Chunrong Fang (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: fangchunrong@nju.edu.cn; Bowen Yu, State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: 201250070@smail.nju.edu.cn; Hongyan Li, School of Big Data and Software Engineering, Chongqing University, Chongqing, China; e-mail: hongyan.li@cqu.edu.cn; Meng Yan, School of Big Data and Software Engineering, Chongqing University, Chongqing, China; e-mail: mengy@cqu.edu.cn; Jianyi Zhou, Huawei Cloud Computing Technologies Co., Ltd., Beijing, China; e-mail: zhoujianyi2@huawei.com; Zhenyu Chen (corresponding author), State Key Laboratory for Novel Software Technology, Nanjing University, Nanjing, China; e-mail: zychen@nju.edu.cn.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2025/2-ART81

<https://doi.org/10.1145/3699598>

respectively. We also explore the performance of generated assertions in detecting real-world bugs, and find LLMs are able to detect 32 bugs from Defects4J on average, with an improvement of 52.38% against the most recent approach EDITAS. Inspired by the findings, we construct a simplistic retrieval-and-repair-enhanced LLM-based approach by transforming the assertion generation problem into a program repair task for retrieved similar assertions. Surprisingly, such a simplistic approach can further improve the prediction accuracy of LLMs by 9.40% on average, leading to new records on both datasets. Besides, we provide additional discussions from different aspects (e.g., the impact of assertion types and test lengths) to illustrate the capacity and limitations of LLM-based approaches. Finally, we further pinpoint various practical guidelines (e.g., the improvement of multiple candidate assertions) for advanced LLM-based assertion generation in the near future. Overall, our work underscores the promising future of adopting off-the-shelf LLMs to generate accurate and meaningful assertions in real-world test cases and reduce the manual efforts of unit testing experts in practical scenarios.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**;

Additional Key Words and Phrases: Unit Testing, Assertion Generation, LLM, AI4SE

#### ACM Reference format:

Quanjun Zhang, Weifeng Sun, Chunrong Fang, Bowen Yu, Hongyan Li, Meng Yan, Jianyi Zhou, and Zhenyu Chen. 2025. Exploring Automated Assertion Generation via Large Language Models. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 81 (February 2025), 25 pages.  
<https://doi.org/10.1145/3699598>

## 1 Introduction

Unit testing has become an essential and mandatory practice during the software development lifecycle [20]. It involves writing unit tests tailored to ensure that individual components, such as methods, classes, or modules, perform as per their intended specifications and usage requirements. Unlike integration and system testing assessing the overall performance of a software system [28], unit testing focuses on validating the basic units, thereby detecting software bugs at an early stage [4]. Besides, effective unit tests enhance production code quality, prevent regressions, facilitate debugging and maintenance, and instill greater confidence in the reliability of the software [24, 40].

However, writing high-quality unit tests manually is a challenging task, which requires testers to possess sufficient testing knowledge and invest considerable time costs. For example, previous work [12] demonstrates that developers usually spend more than 15% of their time in writing unit tests. To this end, significant effort has been devoted to automated generation of unit tests [16, 37]. A unit test typically includes a test prefix, i.e., a sequence of statements that manipulate the unit under test to a specific state, and a test assertion, i.e., an oracle that defines the conditions that should be satisfied in that state [14]. Automated unit test generation tools attempt to generate both test prefixes and assertions [17, 38]. However, since these tools fail to know and understand the intended behavior of generated test prefixes [4, 46], they face several limitations, one of which is the generation of accurate unit assert statements that can properly test the behavior of basic units.

To tackle the **Assertion Generation (AG)** problem and complement existing test generation tools, an increasing number of AG techniques [58, 62] have been proposed. Such techniques can be categorized into three groups: **Deep Learning (DL)**-based, **Information Retrieval (IR)**-based, and integrated ones (abbreviated as *Integration*) that combine IR-based and DL-based methods. In particular, DL-based AG techniques [58] adopt DL models to take a pair consisting of a test prefix and its focal method (i.e., the method under test) as input and generate an

assertion from scratch.<sup>1</sup> In contrast, given an input focal-test, IR-based AG techniques [62] retrieve similar focal-test from a retrieval corpus and adaptively adjust retrieved assertions to obtain the desired outcomes. Integration AG techniques [62] aim to leverage the strengths of both DL and IR approaches to generate appropriate assertions. Despite effectiveness, they still fail to generate a large number of correct assertions due to the following reasons: (1) DL-based AG techniques are limited by the quality and quantity of historical test-assertion pairs, e.g., only a small corpus with only 156,760 samples for *ATLAS*; (2) IR-based AG techniques face difficulties in understanding the semantic differences between focal-test pairs, leading to incorrect modifications of retrieved assertions; (3) Integration AG techniques directly employ DL-based and IR-based methods as independent components to generate assertions, inevitably inheriting two above-mentioned limitations [49].

*This Article.* Recent **Large Language Models (LLMs)** have received much attention in the **Software Engineering (SE)** community, greatly boosting performance in a wide spectrum of code-related tasks [54]. Different from *ATLAS* relying on a limited labeled corpus, such models are first pre-trained by self-supervised training on a large-scale unlabeled corpus (e.g., CodeSearchNet with 2.3 million functions) to derive generic language representation and then transferred to benefit multiple downstream tasks by fine-tuning. Benefiting from the pre-training-and-fine-tuning learning paradigm, LLMs have been successfully applied to various code-related tasks in SE research, such as code summarization [34, 48] and program repair [60, 66]. However, little effort has been spent on understanding the actual benefits LLMs bring when dealing with the automated AG task. In this article, to fill this gap, we investigate the feasibility of leveraging advances in LLMs to generate unit test assertions and facilitate future unit testing studies. In particular, we address the following three **Research Questions (RQs)**: RQ1 answers the inference accuracy performance in generating unit assertions; RQ2 answers the real-world bug detection performance of generated assertions; and RQ3 explores a conceptually simple, generic yet effective AG technique by retrieved assertions.

❶ *Inference Accuracy Evaluation.* In RQ1, we evaluate the five studied LLMs against six state-of-the-art AG approaches on two benchmarks in terms of prediction accuracy. *Results:* LLMs are able to predict unit assertions with remarkable accuracy given focal methods and their corresponding test prefixes, e.g., CodeT5 achieves 58.71% and 48.19% prediction accuracy on two datasets, improving the most relevant baseline *ATLAS* by 86.86% and 122.48%, and the most recent baseline *Integration* by 26.15% and 14.19%, indicating the substantial benefits of using LLMs for AG.

❷ *Bug Detection Evaluation.* In RQ2, to further demonstrate the effectiveness of LLMs in identifying real-world bugs, we combine the assertions generated by LLMs with the corresponding test prefixes generated by a popular unit test generation tool EvoSuite [16] to construct complete test cases, which are then compiled and dynamically executed to detect bugs in a popular fault-detection benchmark Defects4J. *Results:* (1) LLMs are able to detect 32 real-world bugs on average, 30 and 11 more than the baselines NoException and EDITAS; (2) LLMs are able to detect a considerable amount of unique bugs, e.g., UniXcoder detects 22 bugs that EDITAS fails to detect, with an improvement of 100% against EDITAS; (3) the combination of studied LLMs is able to detect 70 bugs, demonstrating the substantial benefits of generating assertions in detecting real-world bugs.

❸ *Enhanced LLM-Based Approach.* Motivated by the observation that the retrieval-based approach identifies a mass of assertions that are identical or similar to ground-truths, we introduce EASE, a retrieval-enhanced LLM-based approach by combining the retrieved assertions and LLMs. *Results:* all studied LLMs are able to achieve new records in terms of prediction accuracy on two datasets, e.g.,

<sup>1</sup>For consistency with prior research [58], we refer to such a pair as *focal-test*.

improving CodeGPT by 19.89% on  $Data_{new}$ . Importantly, the performance indicates the potential of retrieved similar assertions to help LLMs generate more accurate unit assertions.

Besides, we conduct some extended experiments to provide additional discussions about LLM-based AG research from different aspects.

① *Impact of the Number of Candidates.* We investigate the performance improvement when increasing the number of candidate assertions. *Results:* all studied LLMs are able to achieve significantly higher prediction accuracy with multiple candidate assertions, such as 45.32% with only Top-2 candidates for CodeGPT, marking a 17.05% improvement against the performance with one Top-1 candidate, highlighting the superiority and potential of multiple candidates in the future.

② *Impact of Assertion Types.* We investigate whether LLMs demonstrate superior performance when dealing with specific assertion types. *Results:* LLMs are able to accurately generate a variety of assertion types, e.g., `assertNotNull` and `assertTrue` with more than 50% accuracy on average.

③ *Impact of Focal-Test and Assertion Lengths.* We investigate the practical utility of LLMs in generating assertions across different focal-test and assertion lengths. *Results:* the focal-test and assertion lengths have a crucial influence on the prediction accuracy due to the inherent limitation of the transformer, e.g., accuracy drops from 23.93% to 1.18% for assertions over 100 code tokens.

④ *Comparison with LLM-Based Approaches.* We investigate the performance of selected open-source LLMs against previous LLM-based AG approaches. *Results:* directly fine-tuning off-the-shelf LLMs is able to achieve comparable performance against previous LLM-based approaches, e.g., an accuracy of 66.76% and 72.50% of CodeT5 and CodeLLama on the *ATLAS* dataset.

Finally, based on our findings, we pinpoint some open research challenges and provide practical guidelines for applying LLMs to AG for future studies. Overall, our results lead us to conclude that LLMs are considerably accurate and practical AG approaches, underscoring the substantial advancement of LLMs in reducing the manual effort of testing experts.

*Novelty and Contributions.* To sum up, the main contributions of this article are as follows:

- *New Dimension.* We bridge the gap between the recent advances in LLMs and a crucial software testing problem, i.e., *automated AG*. Our work not only provides a comprehensive evaluation of the recent LLMs but also highlights the potential for leveraging LLMs to resolve the important automated AG problem in the future.
- *Extensive Study.* We carry out the first extensive empirical study on the application of LLMs to automated unit test AG. Our study includes: (1) a systematic comparison between studied LLMs and state-of-the-art techniques, showing that straightforward LLM-based techniques can readily surpass existing approaches; (2) a comprehensive evaluation examining the effectiveness of the generated assertions in identifying real-world bugs; (3) further discussions about impact analysis, including assertion types and multiple assertions.
- *Effective Technique.* We propose a conceptually simple and generic retrieval-and-repair-enhanced LLM-based AG approach, which simply adopts the advance of retrieved assertions and program repair to boost AG performance further.
- *Practical Guidelines.* We provide several practical guidelines on applying LLMs for future AG studies, such as domain LLM-based assertion and evaluation metrics.

*Open Science.* To support the open science community, we release the studied datasets, related scripts, trained models, and experimental results in a replication package for future research [68].

## 2 Background and Related Work

### 2.1 Automated Assertion Generation

*EvoSuite* [16] and *Randoop* [37] have been acknowledged as foundational work in the automated test generation field and provide crucial guidance for the conception and development of

follow-up works. However, Shamshiri et al. [46] observe that such heuristic-based test generation tools exhibit insufficiencies in generating useful and meaningful assert statements. Thus, researchers have proposed various approaches to generate assertions capable of capturing functional bugs. One such approach involves leveraging informal descriptions of the basic units found within their documentation to synthesize assertions [6, 7]. These approaches utilize techniques such as **Natural Language Processing (NLP)**, manually-summarized rules or patterns, and search-based techniques [21, 39, 51, 65]. Overall, while these approaches provide valuable insights into automatically generating test assertions, their effectiveness in handling real-world documentation and identifying functional bugs can be limited, particularly when applied to benchmark datasets like Defects4J [26].

In recent years, researchers have started exploring the effectiveness of DL approaches for generating test assertions based on a mass of historical developer-written test cases. These approaches utilize deep neural networks to learn from the existing test cases and generate assert statements. For example, Watson et al. [58] propose *ATLAS*, an approach that utilizes Neural Machine Translation [27, 50] to generate syntactically and semantically correct unit test assert statements for a given *focal-test*. A focal-test consists of a test prefix and its focal method (i.e., the method under test). Further, inspired by the idea of combining IR and DL, Yu et al. [62] propose a retrieval-augmented deep AG method. They introduce two IR-based approaches, namely IR-based assertion retrieval ( $IR_{ar}$ ) and retrieved-assertion adaptation ( $RA_{adapt}$ ), for generating assertions. The  $IR_{ar}$  approach retrieves the assertion with the highest similarity to the given focal-test, using measures like Jaccard similarity. Although  $IR_{ar}$  may not always retrieve completely accurate assertions,  $RA_{adapt}$  automatically adapts the retrieved assertions by replacing tokens within them. Two strategies are proposed for determining replacement values: a heuristic-based approach ( $RA_{adapt}^H$ ) and a neural network-based approach ( $RA_{adapt}^{NN}$ ). In addition, Yu et al. [62] combine IR and DL techniques and propose an integrated approach, referred to as *Integration*. Based on *Integration*, Sun et al. [49] further propose a retrieval-augmented deep AG method, namely *EDITAS*. *EDITAS* views the assertion from a similar focal-test as a prototype and leverages a neural sequence-to-sequence model to learn the assertion edit patterns used to modify the prototype. In this work, we include all of the six above-mentioned approaches from three different categories to evaluate the performance of LLMs.

The literature has also witnessed the emergence of several studies [33, 34, 52] delving into the utilization of LLMs in AG. For instance, Mastropaolo et al. [33, 34] initially pre-train a T5 model [43] with natural language English text and Java programming languages, followed by fine-tuning it to support four tasks: bug-fixing, code mutant injection, AG, and code summarization. These studies explore the potential of transfer learning in code-related tasks, particularly pre-training T5 from scratch. However, our study investigates the capability of five off-the-shelf LLMs in AG, involving prediction accuracy on two datasets, bug detection evaluation on Defects4J, an enhanced approach, and impact analysis. Recently, Nashid et al. [35] proposed CEDAR, a Codex-based approach to apply retrieval-based demonstration selection strategy for program repair and AG. Our work and CEDAR are fundamentally distinct regarding their research objects and learning paradigms and utilized LLMs. CEDAR designs a prompt engineering approach by querying the black-box LLM Codex with few-shot learning, while our work is an empirical study by fine-tuning five open-source LLMs with assertion datasets.

## 2.2 LLM

Our work is inspired by the success of LLMs in the SE domain. In this section, we first introduce some typical LLMs by model architectures and then discuss the application in code-related tasks.



The existing LLMs are built on top of the encoder-decoder transformer architecture [55], which can be classified into three main categories: encoder-only, decoder-only, and encoder-decoder models. Among them, encoder-only LLMs, such as CodeBERT [15] and GraphCodeBERT [23], employ a bidirectional transformer during pre-training, allowing each token to attend to every other token. However, for generation tasks, encoder-only models require an additional decoder that is initialized from scratch and does not benefit from pre-training. Besides, decoder-only LLMs, like GPT [8], are pre-trained using unidirectional language modeling, where tokens can only attend to previous tokens and themselves to predict the next token. While decoder-only models excel in auto-regressive tasks like code completion, the unidirectional framework may be sub-optimal for understanding tasks. On the contrary, encoder-decoder LLMs, such as T5 [34], are well-suited for both understanding (e.g., code classification [32]) and generation tasks (e.g., program repair [67]). These models typically employ denoising sequence-to-sequence pre-training objectives, where the source input is corrupted, and the decoder is tasked with recovering the original sequence.

In this work, we select five different LLMs from all the above categories to provide reliable results in our analysis, including CodeBERT [15], GraphCodeBERT [23], CodeT5 [57], UniXcoder [22], and CodeGPT [32].

Recently, LLMs have been utilized and demonstrated their potential across a wide range of code-related tasks, such as program repair [60, 63], vulnerability repair [19, 59], vulnerability detection [18, 47], code review [30, 54], and unit test generation [45, 61, 64]. For example, Schafer et al. [45] propose TestPilot that iteratively queries ChatGPT to generate unit tests with a prompt containing error messages when dynamic executing previously generated tests. Xia et al. [60] conducted an empirical study to evaluate the performance of several LLMs in fixing real-world software bugs. Besides, Steenhoek et al. [47] present an extensive evaluation of recent LLMs for detecting software vulnerabilities. However, there has been a lack of research focused on evaluating the performance of LLMs specifically for the unit AG task. To address this gap, our article undertakes a comprehensive and in-depth analysis, comparing the capabilities of LLMs with existing AG techniques. More importantly, we aim to provide valuable insights and guidance in the field of AG, paving the way for future advancements and improvements in this community.

### 3 Study on Automated Assertion Generation

#### 3.1 Overall Framework

Figure 1 illustrates the overall workflow of existing automated AG approaches equipped with LLMs, and it is detailed as follows. *In the model training phase*, LLMs are first pre-trained with millions of code snippets from open-source projects and then fine-tuned with labeled focal-test and assertion pairs. Overall, LLMs are composed of an encoder stack and a decoder stack, culminating in a linear layer with softmax activation. Initially, LLMs perform a word embedding to generate representation vectors for tokenized focal-test functions. Then, LLMs feed the vectors to an encoder stack to derive the hidden state, which is further passed into a decoder stack. Finally, the output of the decoder stack is fed into a linear layer with softmax activation to generate the probability distribution of the vocabulary. *In the model inference phase*, after LLMs are well trained, given a focal-test pair, the beam search strategy is leveraged to generate the candidate assertions as prediction results based on the probability distribution of the vocabulary. The generated candidate assertion may be verified by the testing experts for deployment in the unit testing pipeline.

#### 3.2 RQs

In this work, we attempt to address the following three RQs:

*RQ1:* How do LLMs compare to existing AG techniques in terms of inference accuracy?

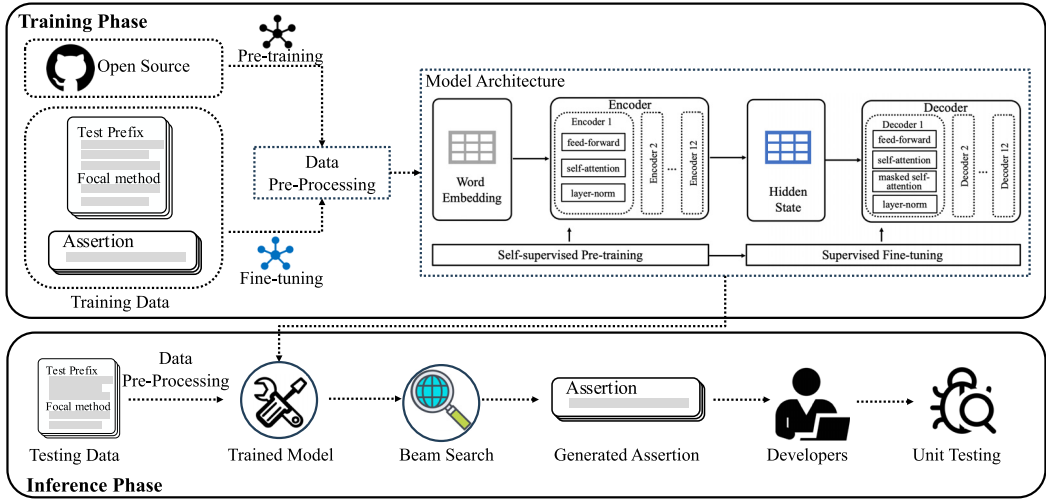


Fig. 1. An overview architecture of LLM-based AG.

RQ2: Can the assertions generated by LLMs effectively detect real-world bugs?

RQ3: Can we enhance the effectiveness of existing LLM-based techniques via retrieved assertions?

With RQ1, we aim to evaluate the performance of LLMs and state-of-the-art AG techniques in terms of accurate AG. We compare the target approaches in a large held-out set of developer-written unit tests collected from real-world projects based on the prediction accuracy. With RQ2, we investigate the bug-detection capability of the generated assertions by evaluating their ability to identify bugs on a set of buggy programs. With RQ3, we attempt to propose a retrieval-enhanced LLM-based approach by combining the retrieved assertions and LLMs, motivated by the retrieval-and-repair-based approach is able to identify a large number of assertions that are identical or similar to ground-truths. We seek to explore whether such a conceptually simple approach can further enhance the performance of LLM in AG. This evaluation process provides insights into the practical applicability and reliability of the generated assertions, allowing us to assess their effectiveness in detecting bugs and their potential for improving software quality. Overall, our study encompasses a comprehensive evaluation of LLMs and AG techniques involving their assertion inference performance, bug-detection capabilities, and the design of a generic retrieval-and-repair-enhanced LLM-based AG approach. In this article, we specifically focus on the Java Junit testing framework as it is widely used in software testing [58, 62].

### 3.3 Dataset

**3.3.1 To Answer RQ1.** In this article, we utilize two datasets, namely  $Data_{old}$  and  $Data_{new}$ , which are introduced by Watson et al. [58] and Yu et al. [62], respectively. These datasets are publicly available and have been widely used in the field of AG research [49, 58, 62].

(1)  $Data_{old}$  dataset is derived from the original dataset utilized by ATLAS. To construct  $Data_{old}$ , a pool of 2.5 million test methods from GitHub is initially collected, which includes test prefixes and their corresponding assertion statements. Each test method in  $Data_{old}$  has been concatenated with a focal method, which represents the production code under test. To ensure the quality and suitability of the dataset, two preprocessing steps are applied: (1) test methods with token lengths exceeding 1K are excluded from the dataset; (2) assertions containing unknown tokens that are not present in the focal-test and the vocabulary are filtered out, resulting in 156,760 samples in total.



Table 1. Detailed Statistics of Each Assertion Type in  $Data_{new}$  and  $Data_{old}$ 

| AssertType   | Total  | Equals       | True        | That        | NotNull    | False      | Null     | ArrayEquals | Same     | Other       |
|--------------|--------|--------------|-------------|-------------|------------|------------|----------|-------------|----------|-------------|
| $Data_{old}$ | 15,676 | 7,866 (50%)  | 2,783 (18%) | 1,441 (9%)  | 1,162 (7%) | 1,006 (6%) | 798 (5%) | 307 (2%)    | 311 (2%) | 2 (0%)      |
| $Data_{new}$ | 26,542 | 12,557 (47%) | 3,652 (14%) | 3,532 (13%) | 1,284 (5%) | 1,071 (4%) | 735 (3%) | 362 (1%)    | 319 (1%) | 3,030 (11%) |

(2) Excluding assertions containing unknown tokens can oversimplify the problem of generating assertions, potentially constraining the dataset’s capacity to accurately represent real-world data distributions and thereby impacting the validity of experimental conclusions. To address this limitation, Yu et al. [62] constructed an expanded dataset called  $Data_{new}$ . This dataset incorporates cases that are initially excluded from  $Data_{old}$  due to the presence of unknown tokens. The inclusion of these additional cases (108,660 samples) brings the total count of data items in  $Data_{new}$  to 265,420.

In this article, we strictly follow the replication package provided by previous studies [49, 62] and divide the  $Data_{old}$  and  $Data_{new}$  datasets into training, validation, and test sets using an 8:1:1 ratio. Table 1 provides statistics of the testing sets and their distribution across various assertion types.

**3.3.2 To Answer RQ2.** Defects4J [26] is a widely-used benchmark for evaluating software bug detection and automated program repair approaches. It consists of 835 bugs collected from 17 real-world Java projects. Following previous work [31], we utilize Defects4J version 2.0.0 to assess the bug-detection performance of AG techniques and LLMs. Each bug in Defects4J is associated with an issue tracker and is fixed in a single commit by modifying the source code. This results in two program versions for each bug: a buggy version and a bug-fixed version. The bug-fixed version is created based on a manually minimized patch and passes all test cases. In contrast, the buggy version fails at least one test case, and these failing test cases are deterministic. To evaluate test suites on specific buggy or bug-fixed program versions, Defects4J provides a supporting framework. The only difference between a buggy program version and its corresponding bug-fixed program version is the minimal patch that fixes the bug. Therefore, a test case that fails on the buggy version but passes on the bug-fixed version can be attributed to triggering the specific software bug.

**3.3.3 To Answer RQ3.** In RQ3, we propose a conceptually simple retrieval-and-repair-enhanced LLM-based approach, namely EASE, by combining the retrieved similar assertions and program repair. We expect our proposed method to have good generalization. To this end, our primary focus centers on assessing the effectiveness of EASE in the context of inference accuracy evaluation. We conduct this evaluation experiment using both the  $Data_{old}$  and  $Data_{new}$  datasets. Besides, due to space limitations, we do not delve into the bug detection performance of EASE within this study.

### 3.4 Baselines

To address the above-mentioned RQs, we compare five LLMs with six state-of-the-art AG techniques, including sequence-to-sequence-based, retrieval-based, and retrieval-enhanced learning-based ones. First, we include *ATLAS*, the first and classical AG technique that utilizes a sequence-to-sequence model to generate assertions from scratch. *ATLAS* is the most relevant baseline as both *ATLAS* and studied LLMs consider AG as a neural machine translation task on top of the encoder-decoder transformer architecture [10, 53]. Second, we incorporate three existing retrieval-based AG techniques:  $IR_{ar}$ ,  $RA_{adapt}^H$ , and  $RA_{adapt}^{NN}$ . Finally, we consider one state-of-the-art retrieval-enhanced learning-based approach *Integration*, and its most recent follow-up *EDITAS*, detailed in Section 2.

### 3.5 Model Selection

In our experiment, we select LLMs based on the following specific requirements. (1) *Sufficient code corpus*. First, the selected LLMs should be pre-trained on a sufficiently large code corpus,

Table 2. The Detailed Information of Selected Open Source LLMs in Our Experiment

| Model        |                 | CodeBERT   | GCodeBERT  | UniXcoder  | CodeT5                 | CodeGPT    |
|--------------|-----------------|------------|------------|------------|------------------------|------------|
| Open Source  |                 | ✓          | ✓          | ✓          | ✓                      | ✓          |
| Programming  |                 | ✓          | ✓          | ✓          | ✓                      | ✓          |
| Archite      | Encode-Only     | ✓          | ✓          |            |                        |            |
|              | Decoder-Only    |            |            |            |                        | ✓          |
|              | Encoder-Decoder |            |            | ✓          | ✓                      |            |
| Organization |                 | HIU        | SYU        | SYU        | Salesforce             | Microsoft  |
| Model Size   |                 | 125M       | 125M       | 125M       | 220M                   | 124M       |
| Pre-training |                 | CSearchNet | CSearchNet | CSearchNet | CSearchNet<br>BigQuery | CSearchNet |

HIT denotes Harbin Institute of Technology; SYU denotes Sun Yat-sen University; GCodeBERT denotes the GraphCodeBERT model; and CsearchNet denotes the CodeSearchNet dataset.

as our focus is on generating unit assertions. Thus, models such as T5 [43] and GPT-2 [42] that are primarily designed for NLP tasks are excluded from our evaluation. (2) *Model availability*. The selected LLMs should be publicly accessible as we need to fine-tune them in our experiment. Thus, models such as Codex [9] are excluded from our evaluation because their models are not publicly released. (3) *Dataset accessibility*. The training datasets of selected LLMs should be released, allowing for an inspection of their overlap with our AG datasets, i.e., the dataset leakage issue of LLMs [69]. Thus, models such as Code LLaMA [44] are excluded as their datasets are closed-source. Table 2 presents the five LLMs evaluated in our experiment. Overall, the utilized models are from different architectures (i.e., encoder-decoder, encoder-only, and decoder-only) and organizations (e.g., Microsoft and Salesforce) to obtain diverse evaluation scenarios. All models represent state-of-the-art and are publicly available on the Hugging Face, which is the largest open-source platform to host and deploy large models, summarized as follows.

—*CodeBERT* [15] is a bi-modal LLM designed for both programming language and natural language understanding. It utilizes a hybrid objective function during pre-training, combining standard **Masked Language Modeling (MLM)** [13] and *replaced token detection* [11]. The MLM aims to predict the original tokens that have been masked out, while the replaced token detection focuses on predicting whether a token is an original token or a replaced token.

—*GraphCodeBERT* [23] takes into account the inherent structure of code on top of the BERT architecture by representing the semantic-level structure of code and capturing the relationships between variables. In addition to the MLM in CodeBERT, GraphCodeBERT further incorporates two structure-aware pre-training tasks. The first task is *data flow edge prediction*, where the model learns to predict the edges in the data flow graph, thus enabling it to learn representations from the code structure. The second task focuses on *aligning representations* between the source code and its corresponding code structure, ensuring consistency between different parts of the code.

—*UniXcoder* [22] is a unified cross-modal LLM designed to enhance both code understanding and generation capabilities. In particular, UniXcoder introduces two novel pre-training tasks to further refine its semantic embedding. The first task is *Multi-modal Contrastive Learning*, where positive examples consist of the same input with different hidden dropout masks, and negative examples are

representations from other instances in the same batch. The second task is *Cross-modal Generation*, which requires the model to generate comments that describe the function of the code.

—*CodeT5* [57] adopts the T5 architectures and incorporates token-type information specific to the source code. In addition to a common denoising pre-training task, CodeT5 further leverages the semantics conveyed by identifiers assigned by developers. To capture this information, CodeT5 includes two identifier-related pre-training tasks. The first task, called *identifier tagging*, aims to classify whether a given code token is an identifier or not. The second task, *masked identifier prediction*, involves corrupting the input sequence by masking all the identifiers in the code segment.

—*CodeGPT* [32] adopts the same model architecture and pre-training objective as GPT-2 [41], which was initially designed for natural language. Specifically, CodeGPT consists of 12 layers of transformer decoders. The pre-training objective of GPT-2, called *auto-regressive language modeling*, involves predicting the next token in a sequence conditioned on the previous tokens and itself.

### 3.6 Evaluation Metrics

For RQ1, we evaluate the quality of the generated assertions through the accuracy measurement, following the methodology employed in prior studies [58, 62]. Accuracy serves to evaluate the correctness of the generated assertions. For RQ2, we evaluate the effectiveness of the generated assertions in bug detection using the BugFound metric, as suggested by Liu et al. [31]. A higher BugFound value corresponds to improved bug detection performance. For RQ3, our focus is solely on assessing the inference correctness of the proposed retrieval-and-repair-enhanced LLM-based approach, for which we employ the accuracy metric. We briefly describe each metric as follows.

① *Accuracy*. Accuracy is defined as the percentage of the generated assertions that exactly match the reference assertions. Accuracy can be considered as the lower bound since the different assertions can be semantically the same but are written differently using the different Junit APIs.

② *BugFound*. BugFound refers to the total number of bugs that can be detected by the generated assertions. It serves as a metric for assessing the recall of an AG approach in terms of bug finding. BugFound provides insights into the effectiveness of the approach in uncovering software bugs.

### 3.7 Implementation Details

Our experiments are built on the PyTorch framework [2]. We employ the Hugging Face implementation [1] of the studied LLMs and all hyperparameters are taken from default values. It is noteworthy that some LLMs come in different architectural sizes, such as CodeT5-base and CodeT5-large. Guided by earlier research [43, 63], we chose the base version as a starting point because of its practicality and effectiveness, which is often comparable to its larger counterpart. We configure the batch size to 8, maximum input length to 512, maximum output length to 256, and learning rate to  $2e-5$ . All training and evaluations are performed on one Ubuntu 20.04 server with four NVIDIA GeForce RTX 3090 GPUs.

## 4 Results and Analysis

### 4.1 RQ1: Assertion Inference Accuracy

*Motivation*. Assertions play a central role in SE, significantly improving code comprehensibility, maintainability, and security. Currently, notable advancements have been made in AG techniques, and LLMs have emerged as powerful tools for processing code language, offering promising prospects in this field. However, in order to fully exploit the potential of LLMs, a comprehensive evaluation of their performance in assertion inference accuracy becomes imperative. Hence, it becomes pertinent to pose the intriguing but vital research question: *Can LLMs generate assertions accurately, and is their performance comparable to the current state-of-the-art techniques?*

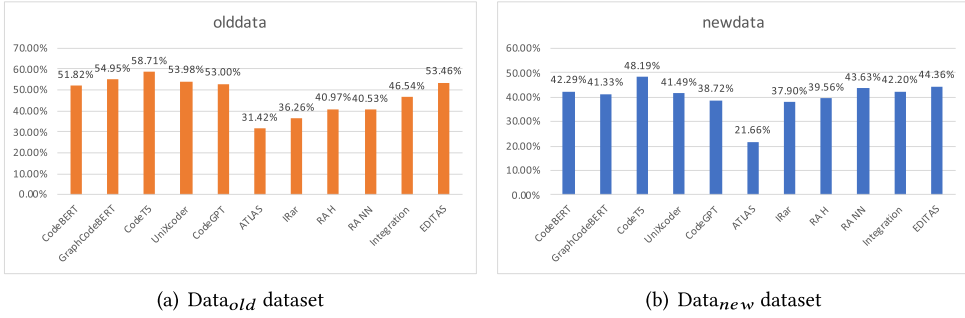


Fig. 2. The experimental results of LLMs and state-of-the-arts in AG.

**Experimental Design.** To answer RQ1, we evaluate the performance of five studied LLMs and six state-of-the-art AG techniques, including *ATLAS*, *IRar*,  $RA_{adapt}^H$ ,  $RA_{adapt}^{NN}$ , *Integration*, and *EDITAS*. To ensure a fair comparison, we provide the same set of test inputs to all the methods under evaluation, and we employ the same training set to train/implement AG methods or fine-tune the LLMs, respectively. Each method takes the focal-test as input and generates the corresponding assertion as output. Finally, we report the prediction accuracy results of all baselines and LLMs. Following existing studies [49, 62], we reuse the released results from their work instead of re-implementing or re-running them ourselves, thereby reducing potential risk.

**Results.** Figure 2 presents the comparison results of LLMs and state-of-the-art AG approaches in terms of prediction accuracy. Overall, LLMs are able to achieve a prediction accuracy of 48.45% on average, outperforming baselines with an improvement of 21.50%. In particular, when compared with the most relevant baseline *ATLAS*, LLMs achieve a prediction accuracy of 38.72%–48.19% on the *Data<sub>new</sub>* dataset, with an accuracy improvement of 78.76%–122.48%, respectively. Similar performance can be observed in the *Data<sub>old</sub>* dataset, and the improvement achieves 64.93%–86.86%. Considering that both studied LLMs and *ATLAS* treat the AG problem as a neural machine translation task on top of the encoder-decoder architecture, the significant improvement highlights the benefits of LLMs against the default transformer in generating assertions. Besides, when compared with existing retrieval-based approaches, LLMs achieve an average accuracy improvement against *IRar*,  $RA_{adapt}^H$ ,  $RA_{adapt}^{NN}$  by 31.08%, 20.62%, and 14.76% on the two datasets, respectively. Compared with the two most recent retrieval-enhanced learning-based approaches *Integration* and *EDITAS* on the *Data<sub>old</sub>* dataset, LLMs improve the average accuracy by 9.51%.

We further analyze the performance among different LLMs and find that CodeT5 always achieves optimal performance. The improvement against CodeBERT, GraphCodeBERT, UniXcoder, and CodeGPT reaches 13.95%, 16.60%, 16.15%, 24.46% on the *Data<sub>new</sub>* dataset. Similarly, the improvement on the *Data<sub>old</sub>* dataset reaches 13.30%, 6.84%, 8.76%, and 10.77%, respectively. We think the possible reason can be attributed to its model architecture and pre-training datasets. Specifically, CodeT5 employs an encoder-decoder architecture, which has proven effective for code generation tasks in prior work. Besides, CodeT5 is pre-trained with both CodeSearchNet and BigQuery datasets, larger than that of other LLMs. In contrast, encoder-only models, like CodeBERT, need a separate decoder for generation tasks, which starts from scratch and does not leverage the advantages offered by the pre-training dataset. More importantly, as a representative LLM, CodeT5 consistently significantly outperforms all compared approaches on the two datasets with an average improvement of 39.65% (104.67% for *ATLAS*, 44.53% for *IRar*, 32.56% for  $RA_{adapt}^H$ , 26.73% for  $RA_{adapt}^{NN}$ , 20.17% for *Integration*, 9.23% for *EDITAS*), demonstrating the superior performance of CodeT5 in AG.

*Answer to RQ1:* Our comparison results reveal that (1) LLMs are able to achieve remarkable performance in automated AG with 51.82%–58.71% and 38.72%–48.19% prediction accuracy on  $Data_{old}$  and  $Data_{new}$ , respectively; (2) compared to the state-of-the-art AG techniques, LLMs achieve better prediction performance with an improvement accuracy of 29.60% and 12.47% on average; (3) CodeT5 substantially outperforms all other LLMs and all baselines on both benchmarks with 13.85% and 39.65% higher prediction accuracy on average.

## 4.2 RQ2: Bug Detection Capabilities

*Motivation.* Evaluating the performance of AG technologies and LLMs only based on the accuracy of generated assertions may fall short of addressing the practical goal in software testing, which is to discover software bugs. Assertions are primarily intended to facilitate the identification of potential issues within software units. This realization has driven our desire to delve deeply into experimentation involving LLMs and AG techniques for bug detection. Furthermore, the commonly used AG benchmarks typically assume the presence of developer-written test prefixes, which may not accurately reflect real-world scenarios. Hence, it becomes essential to pose the critical research question: *Do the assertions generated by LLMs effectively identify software bugs in real scenarios?*

*Experimental Design.* To answer RQ2, we utilize the randomized test generation tool EvoSuite to generate test cases for the buggy programs. It's essential to note that EvoSuite-generated test cases may encompass scenarios that involve testing exception behavior, specifically verifying the occurrence of expected exceptions. Given that our focus is on the generation of assertions, we have implemented a filtering mechanism to exclude test cases involving exception testing from our analysis. This filtering step ensures that the test prefixes generated by EvoSuite are exclusively employed for the purpose of generating assertions, aligning with the core objectives of our study. Then, we extract the test prefixes by removing EvoSuite's assertion statements from the test cases. For each of the generated test prefixes, we apply both LLMs and AG techniques to infer a test assertion, respectively. These generated assertions are then combined with the test prefixes to construct complete test cases. To evaluate the effectiveness of the generated assertions in identifying real-world bugs, we execute the complete test cases on both the buggy and the bug-fixed program versions. Following prior work [31], we limit the runtime of EvoSuite for each buggy program to two minutes and query LLMs to generate only one assertion for each test case. Besides, to mitigate the effects of randomness, we repeat the process ten times and report the overall results. Considering the substantial time and computational resources required for compiling and running all test cases, coupled with the observation that EDITAS demonstrates the highest prediction accuracy among all the AG approaches in RQ1, we opt to compare the bug detection performance of five LLMs and EDITAS in our experiments. To ensure a fair comparison, we fine-tune LLMs and construct the retrieval corpus for EDITAS on the same  $Data_{new}$  training dataset. Additionally, we introduce a baseline method called NoException, as proposed by Liu et al. [31]. This baseline method operates as a straightforward oracle generator, expecting that each test prefix will not trigger an exception and does not necessitate the explicit generation of assertion statements. By comparing the performance of the LLMs with NoException, we aim to provide practitioners with insights into the advantages that an LLM can offer compared to an implicit test case oracle.

*Results.* Figure 3 presents the experimental results of different LLMs in terms of the number of detected bugs. Overall, the investigated LLMs are able to detect 32 real-world bugs by the generated unit assertions on average, 30 and 11 more than the baselines NoException and EDITAS. The improvement achieves 1500.00% and 52.38%, respectively. In particular, CodeBERT, GraphCodeBERT, CodeT5, UniXcoder, and CodeGPT detect 34, 32, 17, 41, and 36 bugs, four of which outperform the state-of-the-art EDITAS, demonstrating the ability of LLMs in detecting software bugs in practice.

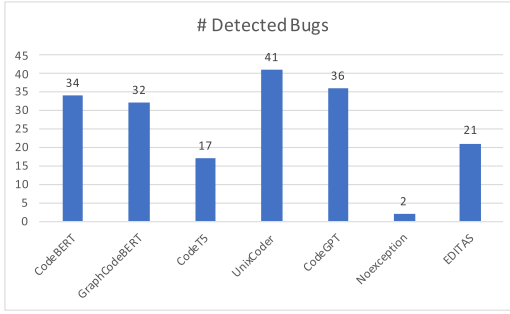


Fig. 3. The bug detection performance of LLMs and state-of-the-arts in AG.

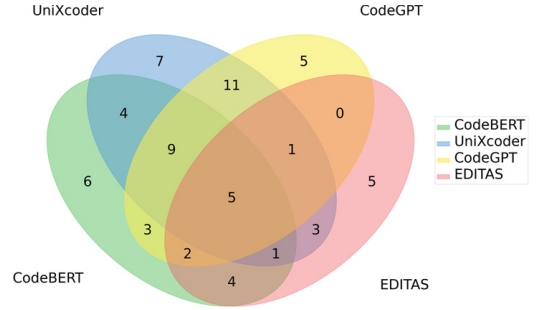


Fig. 4. The overlaps of the bugs are detected by different approaches.

Besides, when comparing the number of detected bugs between different LLMs, we find UniXcoder achieves the optimal performance while CodeT5 performs worst. Considering that CodeT5 achieves the highest prediction accuracy (i.e., 58.71% and 48.19%) among all LLMs on the two datasets, the decline in bug detection performance prompts us to review the difference between these two metrics. We highly recommend the researchers conduct extensive empirical to explore the relationship between the static match-based and the dynamic execution-based metrics in the future.

To further investigate what extent LLMs complement existing AG techniques, we further calculate the number of overlapping bugs detected by different techniques. One best-performing baseline (i.e., EDITAS) and three best-performing LLMs (i.e., CodeBERT, UniXcoder and CodeGPT) are selected. As shown in Figure 4, CodeBERT, UniXcoder and CodeGPT are able to detect 22, 31, and 25 unique bugs that EDITAS fails to detect, while EDITAS can only detect 9, 11, and 13 unique bugs that the three LLMs fail to detect, with an average improvement of 136.36% against EDITAS. More importantly, the combination of the three LLMs is able to detect 61 bugs, demonstrating these models can be used together to further increase the number of bugs that can be detected correctly. Overall, the overlapping results demonstrate that LLMs are complementary to the best-performing AG technique EDITAS, to increase the number of correctly-detected bugs in the Defects4J benchmark.

*Answer to RQ2:* Our results reveal that (1) five selected LLMs are able to detect 32 real-world bugs on average, with an improvement of 52.38% against the state-of-the-art EDITAS; (2) three best-performing LLMs are able to detect 26 unique bugs on average, with an improvement of 136.36% against state-of-the-art EDITAS; (3) the combination of the three LLMs is able to detect 61 unique bugs, demonstrating the potential of LLMs to detect real-world bugs in unit testing.

#### 4.3 RQ3: Retrieval-and-Repair-Enhanced LLM-Based Approach

*Motivation.* In Section 4.1, we find that the simple retrieval-based approach  $IR_{ar}$  is able to outperform the sequence-to-sequence-based approach  $ATLAS$ , i.e., 36.26% vs. 31.42% and 37.90% vs. 21.66% on the two datasets. This observation indicates that a significant proportion of the retrieved assertions exactly match the ground-truth assertions, highlighting the potential of these similar assertions in AG. Besides, although the *Integration* approach inherits from both retrieval-based and learning-based approaches, the utilization of the two remains isolated. In particular, *Integration* first retrieves a similar assertion and regenerates a new assertion based on  $ATLAS$  from scratch if necessary. Thus, it is an interesting research question to ask: *What is the possibility of directly incorporating these valuable retrieved assertions into the process of AG rather than as a preliminary step to determine if the model should generate new assertions?*



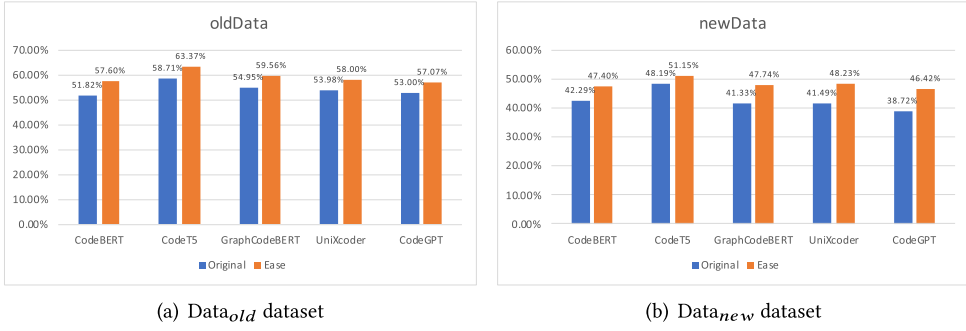


Fig. 5. The experimental results of our retrieval-enhanced LLM-based AG approaches.

**Experimental Design.** We introduce EASE, a retrieval-and-repair-enhanced LLM-based automated AG approach by combining both retrieved assertions and LLMs. Similar to RQ1, we still treat the AG task as an NMT problem, where we feed the model with a focal method and its corresponding test prefix, allowing the model to learn subsequent assertions. Differently, we encode the retrieved assertions as an additional input in the natural language for LLMs. Such a setting aligns more with real-world development scenarios, where developers often refer to similar code snippets from the targeted projects or open-source projects to assist in generating new code snippets, i.e., the plastic surgery hypothesis in program repair [5, 53]. Similar to program repair that generates correct patches for a buggy program by slight modifications, EASE attempts to generate a corrected version of the retrieved assertion with contextual focal-tests. More importantly, unlike *ATLAS*, EASE does not need to generate an entirely new assertion from scratch and only learns a few edit operations by referring to a similar assertion. Besides, unlike *Integration*, EASE does not separate the retrieved assertions from the model’s AG process but instead uses assertions as additional information to enhance the model’s generation capabilities.

**Results.** The results are presented in Figure 5. Overall, EASE consistently achieves better perfect prediction performance across all five LLMs and two datasets, achieving an average accuracy ranging from 48.45% to 53.65%, marking a 10.75% improvement. Compared on *Data<sub>old</sub>*, the improvement with EASE reaches 11.15% for CodeBERT, 8.39% for GraphCodeBERT, 7.94% for CodeT5, 7.45% for UniXcoder, and 7.68% for CodeGPT, respectively. Besides, similar performance can be observed on *Data<sub>new</sub>*. The improvement in prediction accuracy reaches 12.08% for CodeBERT, 15.51% for GraphCodeBERT, 6.14% for CodeT5, 16.24% for UniXcoder, and 19.89% for CodeGPT, respectively. Such improvement highlights that similar assertions from the simple retrieval-based approach are useful and can be repaired to benefit the AG task by different LLMs. In the future, we recommend researchers to further explore the capabilities of advanced AG techniques that combine similar retrieved assertions and LLMs to generate more accurate unit assertions.

**Answer to RQ3:** Our comparison results demonstrate that, by combining LLMs and retrieved assertions, EASE is able to further boost existing AG approach performance, leading to new records on prediction accuracy for all LLMs, e.g., 51.15% for CodeT5 on *Data<sub>new</sub>*.

## 5 Discussion

### 5.1 Impact of the Number of Candidates

**Design.** Following the existing AG work [58, 62], we report the performance of LLMs with a beam size  $k = 1$  that returns only one possible candidate assertion. In related code generation tasks that have been extensively researched, hundreds or even thousands of candidates can be generated by

Table 3. The Experimental Results of LLMs with Different Values of Beam Size ( $k$ ) in AG

| Model         | $k = 1$     | $k = 2$             | $k = 3$             | $k = 4$             | $k = 5$             | $k = 10$            | $k = 30$            | $k = 50$            |
|---------------|-------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|---------------------|
| CodeT5        | 48.19%<br>- | 55.50%<br>(↑ 15.17) | 58.33%<br>(↑ 21.04) | 59.97%<br>(↑ 24.44) | 61.02%<br>(↑ 26.62) | 63.36%<br>(↑ 31.48) | 64.34%<br>(↑ 33.51) | 64.72%<br>(↑ 34.30) |
| CodeGPT       | 38.72%<br>- | 45.32%<br>(↑ 17.05) | 48.61%<br>(↑ 25.54) | 50.57%<br>(↑ 30.60) | 52.09%<br>(↑ 34.53) | 55.94%<br>(↑ 44.47) | 60.28%<br>(↑ 55.68) | 61.87%<br>(↑ 59.79) |
| CodeBERT      | 42.99%<br>- | 48.65%<br>(↑ 13.17) | 51.21%<br>(↑ 19.12) | 52.80%<br>(↑ 22.82) | 53.90%<br>(↑ 25.38) | 56.84%<br>(↑ 32.22) | 60.23%<br>(↑ 40.10) | 61.65%<br>(↑ 43.41) |
| GraphCodeBERT | 41.33%<br>- | 47.78%<br>(↑ 15.61) | 50.36%<br>(↑ 21.85) | 51.81%<br>(↑ 25.36) | 53.07%<br>(↑ 28.41) | 56.10%<br>(↑ 35.74) | 59.75%<br>(↑ 44.57) | 61.66%<br>(↑ 49.19) |
| UniXcoder     | 41.49%<br>- | 47.74%<br>(↑ 15.06) | 50.32%<br>(↑ 21.28) | 51.95%<br>(↑ 25.21) | 53.11%<br>(↑ 28.01) | 56.21%<br>(↑ 35.48) | 59.84%<br>(↑ 44.23) | 61.20%<br>(↑ 47.51) |

↑ denotes performance improvement of LLMs at various  $k$  values against the performance at  $k = 1$ .

some mature approaches to increase the prediction performance, such as 1,000 candidate patches per bug in CIRCLE [63]. In fact, there is a trade-off between the number of generated candidates and the prediction accuracy, which has been overlooked in previous AG studies. On the one hand, in cases where there are constraints on testing resources and limited capacity for manual assessment of generated candidate assertions, it is clearly more practical and efficient to focus on inspecting a smaller subset of candidate assertions, such as the Top-1 and Top-5, which are highly likely to be correct. On the other hand, when automated inspection is available (e.g., using static syntax checking and dynamic execution to filter invalid assertions), testers can filter the incorrect candidate assertions automatically without manual inspection. In such a scenario, setting a higher beam size value allows testers to consider more candidate assertions, potentially leading to a higher prediction accuracy. Thus, in this section, we aim to investigate the prediction accuracy of LLMs with different beam size values, so as to explore how they perform in different practical scenarios.

**Results.** Table 3 displays the prediction accuracy of LLMs across various beam size settings.<sup>2</sup> As detailed in Section 4.1, except for CodeGPT, the LLMs consistently achieve comparable or superior performance compared to EDITAS when generating Top-1 candidate assertion (i.e., beam size  $k = 1$ ). Notably, when the beam size value increases to 5, the prediction accuracy of all LLMs experiences a substantial improvement, ranging from 25.38% to 28.41%. In other words, testers can correctly generate over 50.00% of the desired results for the *Data<sub>new</sub>* by manually inspecting only the Top-5 candidate assertions recommended by the LLMs. Furthermore, irrespective of the specific LLM employed, an enhancement in Top- $k$  prediction accuracy is consistently observed as larger beam size values are set. For instance, the prediction accuracy of UniXcoder increases from 53.11% to 61.20% when the number of candidate assertions ( $k$ ) expands from 5 to 50. This observation serves as an inspiration for future researchers to develop automated inspection methods for assessing the correctness of generated assertions. Automated inspection can underscore the substantial advantages of utilizing larger beam sizes and benefit its adoption in practice, discussed in Section 6.

**Summary:** The results demonstrate significantly better performance with more candidate assertions (e.g., one additional candidate results in an improvement of 17.05% for CodeGPT) and inspire future research on multiple assertion filtering approaches, which can be orthogonal to, yet combined with most existing AG approaches.

<sup>2</sup>In Section 5, we only present experimental results on the recent *Data<sub>new</sub>* benchmark due to page limitations.

Table 4. Detailed Statistics of Different LLMs for Each Assert Type

| Approach      | Total        | AssertType  |             |             |           |           |           |             |           |             |
|---------------|--------------|-------------|-------------|-------------|-----------|-----------|-----------|-------------|-----------|-------------|
|               |              | Equals      | True        | That        | NotNull   | False     | Null      | ArrayEquals | Same      | Other       |
| CodeT5        | 12,791 (48%) | 5,916 (47%) | 1,692 (46%) | 1,532 (43%) | 816 (64%) | 580 (54%) | 404 (55%) | 178 (49%)   | 169 (53%) | 1,504 (50%) |
| CodeGPT       | 10,276 (39%) | 4,718 (38%) | 1,344 (37%) | 1,185 (34%) | 653 (51%) | 466 (44%) | 348 (47%) | 130 (36%)   | 125 (39%) | 1,307 (43%) |
| CodeBERT      | 11,224 (42%) | 5,076 (40%) | 1,538 (42%) | 1,266 (36%) | 769 (60%) | 517 (48%) | 376 (51%) | 155 (43%)   | 142 (45%) | 1,385 (46%) |
| GraphCodeBERT | 10,969 (41%) | 4,999 (40%) | 1,472 (40%) | 1,244 (35%) | 762 (59%) | 496 (46%) | 390 (53%) | 138 (38%)   | 146 (46%) | 1,322 (44%) |
| UniXcoder     | 11,011 (41%) | 5,032 (40%) | 1,508 (41%) | 1,230 (35%) | 756 (59%) | 515 (48%) | 345 (47%) | 145 (40%)   | 146 (46%) | 1,334 (44%) |
| Average       | 42.40%       | 41.00%      | 41.37%      | 36.56%      | 58.51%    | 48.07%    | 50.70%    | 41.22%      | 45.64%    | 45.23%      |

## 5.2 Impact of Assertion Types

*Design.* Different assertion types can have distinct testing implications, as specific types of assertions might be better suited for particular testing requirements and debugging scenarios. The widely-used Junit testing framework offers a selection of assertion types, and the choice of assertion type can significantly impact code quality, maintainability, and the overall stability of a software system. In this section, to clarify the practical utility of LLMs in handling various assertion types, we conduct an in-depth examination to determine whether LLMs demonstrate superior performance when dealing with specific assertion types. The types are identified by the Junit testing framework.

*Results.* Table 4 presents the effectiveness of LLMs for different types of assertions, where each column represents an assertion type, and each cell shows the number of correctly generated assertions and their corresponding ratios in brackets. Overall, the experimental results show that CodeT5 consistently outperforms other LLMs across various assertion types. This superiority may be attributed to the nature of the generative task, where CodeT5, as a pre-trained encoder-decoder architecture, excels in comparison to encoder-only models like CodeBERT. Besides, among all the assertion types, LLMs exhibit better prediction accuracy for NotNull (58.51%), False (48.07%), and Null (50.70%) assertion types than that of Equals (41.00%) and ArrayEquals (41.22%). This difference in performance can be elucidated by considering the increased demand for code comprehension when generating Equals and ArrayEquals assertions. Since these two types of assertions usually involve multiple objects, accurately identifying which objects require assertion and which should be deemed equal, especially within the context of complex function calls, is a non-trivial task, which requires a precise understanding of the intent and behavior of the focal method, as well as the contextual semantic information of the test prefixes. In contrast, the Null and NotNull assertion types involve a relatively simpler determination-whether a single object is null or not. Considering the wide use of Equals assertion type and its powerful assertion capability, there is still room for further enhancement of LLM-based AG approaches at this stage.

*Summary:* The results demonstrate that LLMs excel at handling some assertion types, particularly NotNull, while it is challenging to generate specific types of assertions like ArrayEquals.

## 5.3 Impact of Test Lengths and Assertion Lengths

*Design.* The size of the input sequence is a pivotal factor in the assertion inference phase of LLMs. Longer input sequences invariably lead to an expanded vocabulary and a broader search space. As demonstrated in Section 4.1, LLMs have the capacity to generate a substantial volume of correct assertions. A striking example is CodeT5, which achieved an impressive 48.19% Top-1 accuracy in the *Data<sub>new</sub>* dataset. However, it is noteworthy that despite these accomplishments, there remains a significant proportion of focal-tests where LLMs struggle to produce precise results. Thus, in this section, we perform a further investigation to analyze the prediction accuracy of LLMs concerning the length of focal-tests. Moreover, we delve into the effectiveness of LLMs in generating assertions of different lengths. In particular, LLMs may face challenges when tasked with longer assertions, as

Table 5. The Comparison Results of the Five LLMs When Generating Assertions with Different Lengths of Input Focal-Tests

| #Tokens |         | CodeT5 | CodeGPT | CodeBERT | GraphCodeBERT | UniXcoder | Average |
|---------|---------|--------|---------|----------|---------------|-----------|---------|
| ≤ 500   | 0–100   | 43.94% | 35.16%  | 38.67%   | 37.80%        | 37.50%    | 38.61%  |
|         | 100–200 | 49.98% | 40.05%  | 43.44%   | 42.43%        | 42.80%    | 43.74%  |
|         | 200–300 | 51.35% | 41.62%  | 46.20%   | 44.40%        | 45.02%    | 45.72%  |
|         | 300–400 | 54.21% | 43.75%  | 48.19%   | 47.37%        | 48.60%    | 48.42%  |
|         | 400–500 | 50.74% | 42.43%  | 45.62%   | 45.16%        | 43.91%    | 45.57%  |
|         | Average | 50.04% | 40.60%  | 44.42%   | 43.43%        | 43.57%    | 44.41%  |
| >500    |         | 49.29% | 38.98%  | 40.76%   | 41.13%        | 41.57%    | 42.35%  |

Table 6. The Comparison Results of the Five LLMs When Generating Assertions with Different Lengths

| #Tokens |         | CodeT5 | CodeGPT | CodeBERT | GraphCodeBERT | UniXcoder | Average |
|---------|---------|--------|---------|----------|---------------|-----------|---------|
| ≤ 100   | 0–20    | 61.33% | 52.18%  | 56.90%   | 56.56%        | 56.75%    | 56.74%  |
|         | 20–40   | 37.73% | 27.61%  | 30.84%   | 28.92%        | 29.17%    | 30.85%  |
|         | 40–60   | 25.22% | 15.81%  | 14.64%   | 14.33%        | 13.71%    | 16.74%  |
|         | 60–80   | 15.26% | 9.81%   | 6.81%    | 6.81%         | 6.27%     | 8.99%   |
|         | 80–100  | 17.12% | 3.42%   | 3.42%    | 2.74%         | 4.79%     | 6.30%   |
|         | Average | 31.33% | 21.77%  | 22.52%   | 21.87%        | 22.14%    | 23.93%  |
| >100    |         | 4.81%  | 0.74%   | 0.00%    | 0.37%         | 0.00%     | 1.18%   |

the creation of intricate and comprehensive assertions typically necessitates enhanced language comprehension and reasoning capabilities. Gaining insights into the performance of LLMs across different assertion lengths aids testers in understanding their limitations and practical applicability.

*Results.* Tables 5 and 6 provide a detailed comparative analysis of the performance of five LLMs across various focal-test input lengths and assertion output lengths, respectively. In these tables, the initial two columns display the token count, while the subsequent columns showcase the corresponding prediction accuracy. Each cell value is depicted using a color gradient, with dark green indicating the highest accuracy levels and dark red signifying the lowest accuracy levels.

Overall, it can be found from Table 5 that the performance of all LLMs peaks at 300–400 tokens and then begins to decline as the number of tokens decreases or increases. For instance, when considering focal-tests with fewer than 100 tokens, CodeT5, CodeGPT, CodeBERT, GraphCodeBERT, and UniXcoder demonstrate prediction accuracies of 43.94%, 35.16%, 38.67%, 37.80%, and 37.50%, respectively. These values are notably lower 10.27%, 8.59%, 9.52%, 9.57%, and 11.10% than the results observed for focal-tests ranging from 300 to 400 tokens. However, when the focal-test size exceeds 500 tokens, the accuracy of CodeT5, CodeGPT, CodeBERT, GraphCodeBERT, and UniXcoder stands at 49.29%, 38.98%, 40.76%, 41.13%, 41.57%, and 42.35%, resulting in a 6.08% decrease on average compared with the results for focal-tests ranging 300 to 400 tokens. We attribute this phenomenon to two key factors: ① Notably, small-sized focal-tests are often found to contain limited semantic information. For example, some samples feature focal methods that directly return the results of other functions. The inherent challenge here lies in the insufficiency of contextual information, which impedes LLMs’ understanding of the expected behavior of the focal method. Consequently, LLMs face difficulties in generating precise assertions with some small-sized focal-tests. ② To reduce the GPU memory and computation resources, most LLMs frequently adopt a strategy of constraining input size, such as CodeBERT retaining only the initial 512 tokens and truncating the rest of the input. Besides, it is difficult for neural networks to learn the semantic information for

complex focal-tests due to the long-term dependency problem. Consequently, if crucial information related to assertions resides in the truncated portion of a code snippet, LLMs encounter formidable challenges in generating accurate assertions for such complex focal-tests.

For Table 6, it can be found that the AG performance of LLMs generally decreases as the size of the assertion increases. For example, when the length of assertion is less than 20 tokens, CodeT5, CodeBERT, GraphCodeBERT, UniXcoder and CodeGPT have a prediction accuracy of 61.33%, 56.90%, 56.56%, 56.75% and 52.18%, which are 17.12%, 3.42%, 2.74%, 4.79% and 3.42% for assertions with 80–100 tokens, respectively. We have also observed that LLMs exhibit their highest accuracy when generating assertions containing fewer than 100 tokens. As an illustrative example, CodeT5 achieves a 31.33% prediction accuracy for assertions of this size. However, this accuracy notably declines to 4.81% when dealing with assertions exceeding 100 tokens. Similar trends are observed in the performance of other models. The drop in prediction accuracy for other models reaches 26.52% (31.33%–4.81%), 21.03% (21.77%–0.74%), 22.52% (22.52%–0.00%), 21.50% (21.87%–0.37%), and 22.14% (22.14%–0.00%). We think the possible reason lies in the AG process. LLMs usually employ the beam search algorithm to generate candidate assertions as prediction results based on the probability distribution of the vocabulary, i.e., iteratively selecting several probable tokens according to the likelihood estimation score. Therefore, longer assertions may result in a larger search space and more iterative rounds, thereby reducing the LLMs' performance.

*Summary:* The results demonstrate that (1) the performance of LLMs does not degrade as the size of the focal-tests increases due to the trade-off between contextual information and the length of tokens to be processed, and (2) LLMs excel at generating small-sized assertions due to the employed beam search strategy, e.g., a highest accuracy of 56.74% for 0–20 code tokens.

#### 5.4 Comparison with LLM-Based Baselines

*Design.* As discussed in Section 4, following the experimental design of the most recent work EDITAS [49], we select six prior AG approaches from three categories as baselines. The results demonstrate the superior performance of the five involved LLMs against these baselines. Despite that, we notice that there exist some studies leveraging LLMs in generating unit assertions [33–35], as listed in Section 2.1. Thus, in this section, we attempt to investigate how these LLM-based AG approaches perform compared to our selected LLMs. Particularly, Mastropao et al. [33, 34] pre-train a T5 model with a self-supervised task (e.g., filling masked random words) and fine-tune it with small supervised datasets to support four downstream tasks. Nashid et al. [35] designed a prompt-based few-shot learning strategy, CEDAR, and query Codex to perform AG and program repair. However, we fail to directly include these studies as baselines because the datasets used in our work are different from theirs: both Mastropao et al. [33, 34] and CEDAR are evaluated on the dataset from ATLAS, while our work and all involved baselines in Section 3.4 are evaluated on  $Data_{old}$  and  $Data_{new}$ . Although both  $Data_{old}$  and  $Data_{new}$  are built on top of ATLAS,  $Data_{old}$  involves removing some samples, such as those that could not be tokenized in ATLAS, and  $Data_{new}$  involves adding some samples, such as those containing unknown tokens in ATLAS.

Therefore, we conduct an additional experiment by directly fine-tuning off-the-shelf LLMs against prior LLM-based AG approaches on the ATLAS dataset. To compare with Mastropao et al. [33, 34], we select CodeT5, which shares a similar encoder-decoder model architecture and the number of parameters as T5. Besides, to ensure a proper comparison with CEDAR, which utilizes Codex with 12 billion parameters, we choose CodeLlama-7B as the foundation model to generate assertions since Codex is not open-source. To the best of our knowledge, this is the first attempt to fine-tune CodeLlama-7B in AG research, representing the largest LLM in the community so far.

Table 7. Comparison with LLM-Based AG Approaches

| Approaches | CodeT5 | CodeLLama | Mastropao et al. [34] | Mastropao et al. [33] | CEDAR [35] |
|------------|--------|-----------|-----------------------|-----------------------|------------|
| Accuracy   | 66.76% | 72.50%    | 47.00%                | 68.93%                | 75.55%     |

*Results.* Table 7 presents the prediction accuracy of CodeT5, CodeLlama-7B, and LLM-based approaches on the *ATLAS* dataset. We find that CodeT5 achieves 66.76% accuracy, which is significantly better than Mastropao et al. [34] and comparable to Mastropao et al. [33]. It is worth noting that Mastropao et al. [34] is the extended version of Mastropao et al. [33], both employing the same technical design but only utilizing a new version of T5 model and camelCase naming convention. Besides, CodeLlama-7B achieves a comparable or slightly worse performance against CEDAR with an accuracy of 72.50% on *ATLAS*, which is significantly better than Codet5, indicating that fine-tuning larger models tends to yield better accuracy. It is worth noting that the performance of CodeLlama-7B is valuable as CEDAR employs IR to provide six relevant examples to guide Codex in generating assertions, while we directly fine-tune CodeLlama-7B without any retrieval-augmented process. As mentioned in Section 4.3, IR can significantly enhance the prediction results, which is also proven in the original paper of CEDAR, e.g., only 44.41% accuracy of CEDAR with the one-shot setting. Thus, we are confident that CodeLlama-7B is able to achieve better results when equipped with retrieved assertions. Overall, the promising results motivate researchers to further explore the potential of fine-tuning such newly-released billion-level LLMs with the retrieval-argument generation pipeline in the future.

*Summary:* The results demonstrate that directly fine-tuning off-the-shelf LLMs is able to achieve comparable performance against previous LLM-based AG approaches.

## 6 Implication and Guideline

We summarize the following practical guidelines for future LLM-based AG studies.

*More Advanced LLM-Based AG.* In Section 4.1, our results show that the studied LLMs are able to achieve better performance compared with existing AG approaches. For example, CodeT5 predicts unit assertions with 58.71% and 48.19% accuracy, improving the most relevant baseline *ATLAS* by 122.48% and 86.86%. Similar to *ATLAS*, we regard the AG task as an NMT task, which translates the test prefix and its focal method into the corresponding test assertion automatically. However, such a translation approach is universal for many code generation tasks (e.g., program repair), thus without considering the domain knowledge regarding unit testing and assertions in our work. Such observation motivates future researchers to investigate more advanced LLM-based AG techniques by integrating domain-specific information. For example, it is promising to further combine retrieval-based approaches with LLMs, or consider the auxiliary role of assertion types during the process of AG.

*Practical Evaluation Metrics.* As discussed in Section 4.2, in addition to the static match-based metrics (i.e., accuracy), we evaluate the performance of the generated assertions by dynamic compilation and execution. Surprisingly, we observe a performance difference between the two metrics. For example, CodeT5 achieves the best accuracy of 45.86% among all LLMs while only detecting 17 bugs in Defects4J, the lowest among all LLMs. We think this gap is mainly due to the inherent characteristics of these two metrics. On one hand, the static match-based metric directly matches the tokens of the generated assertion and ground truth, thus lightweight enough to evaluate large-scale benchmarks without execution and employed in most previous studies [58, 62]. However, it is initially designed for NLP tasks without considering the important syntactic



and semantic features of codes. On the other hand, the dynamic execution-based metric can assess the correctness of the generated assertion more accurately, while it is quite time-consuming to execute the generated assertions against systems under test. In the future, the community needs large-scale empirical work to validate whether the match-based metric can accurately reflect the bug detection capability of these AG approaches. Besides, we suggest that the relationships between the classical static match-based and the recent dynamic execution-based metrics need to be further explored.

*The Improvement of Multiple Candidates.* In Section 5.1, our study demonstrates that more candidate assertions can lead to significant improvements in prediction accuracy. For example, CodeGPT achieves a prediction accuracy of 38.72% when considering the Top-1 candidate assertion (i.e., beam size  $k = 1$ ). When the beam size value increases to 3, the prediction accuracy is increased to 48.61%, with an improvement of 25.54%. In other words, architects can correctly write 2,625 more unit assertions out of 26,542 testing instances when only manually inspecting the Top-3 candidate assertions recommended by CodeGPT. Such observation motivates future researchers to assess the candidate assertion correctness automatically, highlighting the substantial benefits of a larger beam size. For example, the researchers are recommended to use compilation or dynamic execution to filter out invalid assertions. Furthermore, similar to the patch overfitting research, it is promising to design some assertion correctness assessment approaches to identify assertions that have a high probability of being correct rather than relying on the default beam search algorithm of LLMs.

*The Utilization Strategies of LLMs.* In this work, we adapt off-the-shelf LLMs in the domain of AG by fine-tuning with supervised datasets. The community has witnessed other potential strategies for utilizing LLMs, such as CEDAR, as mentioned in Section 3.4. In fact, CEDAR and our work are orthogonal in their use of LLMs: the former relies on few-shot or zero-shot learning, while the latter relies on fine-tuning. On the one hand, the former does not require additional training, but it usually relies on extremely large-scale foundation models with billion-level or more parameters. The pre-training costs of such LLMs are unaffordable for most researchers and even enterprises, being limited to a few commercial, closed-source products, such as Codex and ChatGPT, thus limiting model selection and may cause data leakage issues. On the other hand, the latter, despite needing fine-tuning, allows researchers to choose much smaller open-source models that can be better adapted to specific domains and effectively address data leakage concerns. For example, CodeT5, with only 220 million parameters in our work, achieves impressive performance against the most recent EDITAS with a much smaller size (only 1.8% of Codex's parameters) and addresses data leakage with a completely open-source model, including model weights and datasets. In the future, it will be quite interesting for researchers to thoroughly compare two orthogonal dimensions in the field of AG.

## 7 Threats to Validity

*Internal validity threats* pertain to factors within our study that could potentially influence our results. To mitigate these threats, we adhere to the default settings utilized in the original paper from the *Integration* method. For selected LLMs, we rely on the official Hugging Face implementations of the models. A second potential internal validity threat arises from the data leakage problem, i.e., the pre-training data for these LLMs may contain the test methods from the two AG benchmarks. This is because LLMs often use public repositories like GitHub repositories up to a specific date as part of their pre-training data [3, 36, 57]. We then query the pre-training datasets (e.g., CodeSearchNet [25]) of all studied LLMs and find such LLMs do not have access to any test cases, including assertions, during pre-training. Thus, we are confident that the data leakage is not a key point to our conclusion.

*Construct validity threats* in our study primarily revolve around the possibility of experimental bias. To counteract this, we have taken several measures. Firstly, we employ prediction accuracy, a metric widely employed in prior research [58, 62], to measure the accuracy of assertion inference. However, such a static, match-based metric may not fully reflect real-world practices and can not explicitly demonstrate the effectiveness of generated assertions in uncovering real-world bugs. Besides, we introduce a dynamic, execution-based metric, known as BugFound. This metric is better suited for evaluating the effectiveness of generated assertions in bug detection, offering a more realistic assessment. By incorporating these metrics, we aim to provide a more comprehensive evaluation of the generated assertions, thereby mitigating potential threats to construct validity.

*External validity threats* in our study revolve around the extent to which our findings can be generalized to other datasets or LLMs. In this work, we concentrate on AG for Java language. However, since LLMs are not inherently designed for any specific programming language, we believe that the impact of this threat is relatively minor. In future research, we intend to expand our experiments to include datasets encompassing different programming languages other than Java, thereby further validating the generalizability of our findings. Besides, we experiment with five LLMs from three types of model architectures. It is important to note that larger LLMs have been recently released, such as StarCoder [29] and CodeT5+ [56]. Considering that selected LLMs cover different architectures, organizations, parameter sizes, and pre-training datasets, and previous work [60] has shown larger LLMs usually yield better results, we are confident in extending our findings to newly-released larger LLMs. In the future, we will also attempt to explore these newly-released larger LLMs in AG.

## 8 Conclusion

In this article, we perform the first exploration to investigate the feasibility of LLMs on automated AG. Specifically, we conduct a large-scale study to analyze the effectiveness of AG approaches supported by LLMs involving two assertion datasets. The results demonstrate that LLMs can consistently outperform state-of-the-art techniques with an accuracy of 51.82%–58.71% and 38.72%–48.19%, highlighting the substantial advancement of LLMs in AG. We further analyze the effectiveness of LLMs in identifying real-world bugs and find that LLMs are able to detect 32 real-world bugs on average, and detect a number of unique bugs not identified by existing AG techniques. Surprisingly, we also demonstrate that a simplistic retrieval-and-repair-enhanced LLM-based approach can further boost AG performance. Lastly, our findings reveal various practical guidelines for future LLM-based AG. Overall, our work demonstrates the promising future of using LLMs to generate real-world unit assertions and help under-resourced developers test the basic units of software systems in practice.

## Acknowledgment

The authors would like to thank the anonymous reviewers for their insightful comments.

## References

- [1] Hugging Face. 2023. Retrieved September 28, 2023 from <https://huggingface.co/>
- [2] PyTorch. 2023. Retrieved September 28, 2023 from <https://pytorch.org/>
- [3] Wasi Uddin Ahmad, Saikat Chakraborty, Baishakhi Ray, and Kai-Wei Chang. 2021. Unified pre-training for program understanding and generation. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '21)*. Kristina Toutanova, Anna Rumshisky, Luke Zettlemoyer, Dilek Hakkani-Tür, Iz Beltagy, Steven Bethard, Ryan Cotterell, Tanmoy Chakraborty, and Yichao Zhou (Eds.), ACL, 2655–2668. DOI: <https://doi.org/10.18653/V1/2021.NAACL-MAIN.211>
- [4] Mohammad Moein Almasi, Hadi Hemmati, Gordon Fraser, Andrea Arcuri, and Janis Benefelds. 2017. An industrial evaluation of unit test generation: Finding real faults in a financial application. In *Proceedings of the 39th IEEE/ACM*

- International Conference on Software Engineering: Software Engineering in Practice Track (ICSE-SEIP '17)*. IEEE Computer Society, 263–272. DOI : <https://doi.org/10.1109/ICSE-SEIP.2017.27>
- [5] Earl T. Barr, Yuriy Brun, Premkumar T. Devanbu, Mark Harman, and Federica Sarro. 2014. The plastic surgery hypothesis. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*. Shing-Chi Cheung, Alessandro Orso, and Margaret-Anne D. Storey (Eds.), ACM, New York, NY, 306–317. DOI : <https://doi.org/10.1145/2635868.2635898>
  - [6] Arianna Blasi, Alberto Goffi, Konstantin Kuznetsov, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Sergio Delgado Castellanos. 2018. Translating code comments to procedure specifications. In *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '18)*. Frank Tip and Eric Bodden (Eds.), ACM, New York, NY, 242–253. DOI : <https://doi.org/10.1145/3213846.3213872>
  - [7] Arianna Blasi, Alessandra Gorla, Michael D. Ernst, Mauro Pezzè, and Antonio Carzaniga. 2021. MeMo: Automatically identifying metamorphic relations in Javadoc comments for test automation. *Journal of Systems and Software* 181 (2021), 111041. DOI : <https://doi.org/10.1016/j.jss.2021.111041>
  - [8] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Proceedings of the Annual Conference on Neural Information Processing Systems (NeurIPS '20)*. Retrieved from <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>
  - [9] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from <https://arxiv.org/abs/2107.03374>
  - [10] Matteo Ciniselli, Nathan Cooper, Luca Pascarella, Antonio Mastropaolo, Emad Aghajani, Denys Poshyvanyk, Massimiliano Di Penta, and Gabriele Bavota. 2021. An empirical study on the usage of transformer models for code completion. *IEEE Transactions on Software Engineering* 48, 12 (2021), 4818–4837.
  - [11] Kevin Clark, Minh-Thang Luong, Quoc V. Le, and Christopher D. Manning. 2020. ELECTRA: Pre-training text encoders as discriminators rather than generators. In *Proceedings of the 8th International Conference on Learning Representations (ICLR '20)*. OpenReview.net. Retrieved from <https://openreview.net/forum?id=r1xMH1BtvB>
  - [12] Ermira Daka and Gordon Fraser. 2014. A survey on unit testing practices and problems. In *Proceedings of the 25th IEEE International Symposium on Software Reliability Engineering (ISSRE '14)*. IEEE Computer Society, 201–211. DOI : <https://doi.org/10.1109/ISSRE.2014.11>
  - [13] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT '19)*. ACL, 4171–4186. DOI : <https://doi.org/10.18653/v1/n19-1423>
  - [14] Elizabeth Dinella, Gabriel Ryan, Todd Mytkowicz, and Shuvendu K. Lahiri. 2022. TOGA: A neural method for test oracle generation. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 2130–2141. DOI : <https://doi.org/10.1145/3510003.3510141>
  - [15] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Linjun Shou, Bing Qin, Ting Liu, Daxin Jiang, et al. 2020. CodeBERT: A pre-trained model for programming and natural languages. In *Proceedings of the International Conference on Findings of the Association for Computational Linguistics (EMNLP '20)*. ACL, 1536–1547. DOI : <https://doi.org/10.18653/v1/2020.findings-emnlp.139>
  - [16] Gordon Fraser and Andrea Arcuri. 2011. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (FSE-19) and Proceedings of the 13th European Software Engineering Conference (ESEC-13)*. ACM, New York, NY, 416–419. DOI : <https://doi.org/10.1145/2025113.2025179>
  - [17] Gordon Fraser and Andrea Arcuri. 2013. Whole test suite generation. *IEEE Transactions on Software Engineering* 39, 2 (2013), 276–291. DOI : <https://doi.org/10.1109/TSE.2012.14>
  - [18] Michael Fu and Chakkrit Tantithamthavorn. 2022. LineVul: A transformer-based line-level vulnerability prediction. In *Proceedings of the 19th IEEE/ACM International Conference on Mining Software Repositories (MSR '22)*. ACM, New York, NY, 608–620. DOI : <https://doi.org/10.1145/3524842.3528452>
  - [19] Michael Fu, Chakkrit Tantithamthavorn, Trung Le, Van Nguyen, and Dinh Q. Phung. 2022. VulRepair: A T5-based automated software vulnerability repair. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.), ACM, New York, NY, 935–947. DOI : <https://doi.org/10.1145/3540250.3549098>
  - [20] Patrice Godefroid, Nils Klarlund, and Koushik Sen. 2005. DART: Directed automated random testing. In *Proceedings of the ACM SIGPLAN 2005 Conference on Programming Language Design and Implementation*. Vivek Sarkar and Mary W. Hall (Eds.), ACM, New York, NY, 213–223. DOI : <https://doi.org/10.1145/1065010.1065036>

- [21] Alberto Goffi, Alessandra Gorla, Michael D. Ernst, and Mauro Pezzè. 2016. Automatic generation of oracles for exceptional behaviors. In *Proceedings of the 25th International Symposium on Software Testing and Analysis (ISSTA '16)*. Andreas Zeller and Abhik Roychoudhury (Eds.), ACM, New York, NY, 213–224. DOI: <https://doi.org/10.1145/2931037.2931061>
- [22] Daya Guo, Shuai Lu, Nan Duan, Yanlin Wang, Ming Zhou, and Jian Yin. 2022. UniXcoder: Unified cross-modal pre-training for code representation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers) (ACL '22)*. Smaranda Muresan, Preslav Nakov, and Aline Villavicencio (Eds.), ACL, 7212–7225. DOI: <https://doi.org/10.18653/v1/2022.acl-long.499>
- [23] Daya Guo, Shuo Ren, Shuai Lu, Zhangyin Feng, Duyu Tang, Shujie Liu, Long Zhou, Nan Duan, Alexey Svyatkovskiy, Shengyu Fu, et al. 2021. GraphCodeBERT: Pre-training code representations with data flow. In *Proceedings of the 9th International Conference on Learning Representations (ICLR '21)*. OpenReview.net. DOI: <https://openreview.net/forum?id=jLoC4ez43PZ>
- [24] Alan Hartman. 2002. Is ISSTA research relevant to industry? In *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA '02)*. ACM, New York, NY, 205–206. DOI: <https://doi.org/10.1145/566172.566207>
- [25] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2019. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. DOI: <http://arxiv.org/abs/1909.09436>
- [26] René Just, Darioush Jalali, and Michael D. Ernst. 2014. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *International Symposium on Software Testing and Analysis (ISSTA '14)*. ACM, New York, NY, 437–440. DOI: <https://doi.org/10.1145/2610384.2628055>
- [27] Nal Kalchbrenner and Phil Blunsom. 2013. Recurrent continuous translation models. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing (EMNLP '13)*. ACL, 1700–1709. DOI: <https://aclanthology.org/D13-1176/>
- [28] Hareton K. N. Leung and Lee J. White. 1990. A study of integration testing and software regression at the integration level. In *Proceedings of the Conference on Software Maintenance (ICSM '90)*. IEEE, 290–301. DOI: <https://doi.org/10.1109/ICSM.1990.131377>
- [29] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: may the source be with you! arXiv:2305.06161. Retrieved from <https://doi.org/10.48550/ARXIV.2305.06161>
- [30] Zhiyu Li, Shuai Lu, Daya Guo, Nan Duan, Shailesh Jannu, Grant Jenks, Deep Majumder, Jared Green, Alexey Svyatkovskiy, Shengyu Fu et al. 2022. Automating code review activities by large-scale pre-training. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '22)*. Abhik Roychoudhury, Cristian Cadar, and Miryung Kim (Eds.), ACM, New York, NY, 1035–1047. DOI: <https://doi.org/10.1145/3540250.3549081>
- [31] Zhongxin Liu, Kui Liu, Xin Xia, and Xiaohu Yang. 2023. Towards more realistic evaluation for neural test oracle generation. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. René Just and Gordon Fraser (Eds.), ACM, New York, NY, 589–600. DOI: <https://doi.org/10.1145/3597926.3598080>
- [32] Shuai Lu, Daya Guo, Shuo Ren, Junjie Huang, Alexey Svyatkovskiy, Ambrosio Blanco, Colin B. Clement, Dawn Drain, Daxin Jiang, Duyu Tang, et al. 2021. CodeXGLUE: A machine learning benchmark dataset for code understanding and generation. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1*. Joaquin Vanschoren and Sai-Kit Yeung (Eds.), Retrieved from <https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/c16a5320fa475530d9583c34fd356ef5-Abstract-round1.html>
- [33] Antonio Mastropaolo, Nathan Cooper, David Nader-Palacio, Simone Scalabrino, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2023. Using transfer learning for code-related tasks. *IEEE Transactions on Software Engineering* 49, 4 (2023), 1580–1598. DOI: <https://doi.org/10.1109/TSE.2022.3183297>
- [34] Antonio Mastropaolo, Simone Scalabrino, Nathan Cooper, David Nader-Palacio, Denys Poshyvanyk, Rocco Oliveto, and Gabriele Bavota. 2021. Studying the usage of text-to-text transfer transformer to support code-related tasks. In *Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering (ICSE '21)*. IEEE, 336–347. DOI: <https://doi.org/10.1109/ICSE43902.2021.00041>
- [35] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*.
- [36] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. A conversational paradigm for program synthesis. arXiv:2203.13474. Retrieved from <https://doi.org/10.48550/arXiv.2203.13474>
- [37] Carlos Pacheco and Michael D. Ernst. 2007. Randoop: feedback-directed random testing for Java. In *Proceedings of the Companion to the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '07)*. ACM, New York, NY, 815–816. DOI: <https://doi.org/10.1145/1297846.1297902>

- [38] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. 2007. Feedback-directed random test generation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE '07)*. IEEE Computer Society, 75–84. DOI : <https://doi.org/10.1109/ICSE.2007.37>
- [39] Rahul Pandita, Xusheng Xiao, Hao Zhong, Tao Xie, Stephen Oney, and Amit M. Paradkar. 2012. Inferring method specifications from natural language API descriptions. In *Proceedings of the 34th International Conference on Software Engineering (ICSE '12)*. Martin Glinz, Gail C. Murphy, and Mauro Pezzè (Eds.), IEEE Computer Society, 815–825. DOI : <https://doi.org/10.1109/ICSE.2012.6227137>
- [40] Strategic Planning. 2002. *The Economic Impacts of Inadequate Infrastructure for Software Testing*. National Institute of Standards and Technology.
- [41] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. 2018. Improving language understanding by generative pre-training. *OpenAI Blog* (2018), 4.
- [42] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever. 2019. Language models are unsupervised multitask learners. *OpenAI Blog* 1, 8 (2019), 9.
- [43] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research* 21 (2020), 140:1–140:67. Retrieved from <http://jmlr.org/papers/v21/20-074.html>
- [44] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from <https://arxiv.org/abs/2308.12950>
- [45] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An empirical evaluation of using large language models for automated unit test generation. *The IEEE Transactions on Software Engineering* 50, 1 (2024), 85–105. DOI : <https://doi.org/10.1109/TSE.2023.3334955>
- [46] Sina Shamshiri. 2015. Automated unit test generation for evolving software. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE '15)*. ACM, New York, NY, 1038–1041. DOI : <https://doi.org/10.1145/2786805.2803196>
- [47] Benjamin Steenhoeck, Md Mahbubur Rahman, Richard Jiles, and Wei Le. 2023. An empirical study of deep learning models for vulnerability detection. In *Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE '23)*. IEEE, 2237–2248. DOI : <https://doi.org/10.1109/ICSE48619.2023.00188>
- [48] Weisong Sun, Chunrong Fang, Yuchen Chen, Quanjun Zhang, Guan hong Tao, Yudu You, Tingxu Han, Yifei Ge, Yuling Hu, Bin Luo, and Zhenyu Chen. 2024. An extractive-and-abstractive framework for source code summarization. *ACM Transactions on Software Engineering and Methodology* 33, 3 (2024), 75:1–75:39. DOI : <https://doi.org/10.1145/3632742>
- [49] Weifeng Sun, Hongyan Li, Meng Yan, Yan Lei, and Hongyu Zhang. 2023. Revisiting and improving retrieval-augmented deep assertion generation. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 1123–1135. DOI : <https://doi.org/10.1109/ASE56229.2023.00090>
- [50] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. 2014. Sequence to sequence learning with neural networks. In *Proceedings of the Annual Conference on Neural Information Processing Systems*, 3104–3112. Retrieved from <https://proceedings.neurips.cc/paper/2014/hash/a14ac55a4f27472c5d894ec1c3c743d2-Abstract.html>
- [51] Shin Hwei Tan, Darko Marinov, Lin Tan, and Gary T Leavens. 2012. @ tcomment: Testing javadoc comments to detect comment-code inconsistencies. In *Proceedings of the IEEE 5th International Conference on Software Testing, Verification and Validation*. IEEE, 260–269.
- [52] Michele Tufano, Dawn Drain, Alexey Svyatkovskiy, and Neel Sundaresan. 2022. Generating accurate assert statements for unit test cases using pretrained transformers. In *Proceedings of the IEEE/ACM International Conference on Automation of Software Test (AST@ICSE '22)*. ACM, New York, NY/IEEE, 54–64. DOI : <https://doi.org/10.1145/3524481.3527220>
- [53] Michele Tufano, Cody Watson, Gabriele Bavota, Massimiliano Di Penta, Martin White, and Denys Poshyvanyk. 2019. An empirical study on learning bug-fixing patches in the wild via neural machine translation. *ACM Transactions on Software Engineering and Methodology* 28, 4 (2019), 19:1–19:29. DOI : <https://doi.org/10.1145/3340544>
- [54] Rosalia Tufano, Simone Masiero, Antonio Mastropaolo, Luca Pascarella, Denys Poshyvanyk, and Gabriele Bavota. 2022. Using pre-trained models to boost code review automation. In *Proceedings of the 44th IEEE/ACM 44th International Conference on Software Engineering (ICSE '22)*. ACM, NEW YORK, NY, 2291–2302. DOI : <https://doi.org/10.1145/3510003.3510621>
- [55] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Annual Conference on Neural Information Processing Systems*, 5998–6008. Retrieved from <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [56] Yue Wang, Hung Le, Akhilesh Gotmare, Nghi D. Q. Bui, Junnan Li, and Steven C. H. Hoi. 2023. CodeT5+: Open code large language models for code understanding and generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '23)*. Houda Bouamor, Juan Pino, and Kalika Bali (Eds.), ACL, 1069–1088. DOI : <https://doi.org/10.18653/V1/2023.EMNLP-MAIN.68>



- [57] Yue Wang, Weishi Wang, Shafiq R. Joty, and Steven C. H. Hoi. 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), ACL, 8696–8708. DOI : <https://doi.org/10.18653/V1/2021.EMNLP-MAIN.685>
- [58] Cody Watson, Michele Tufano, Kevin Moran, Gabriele Bavota, and Denys Poshyvanyk. 2020. On learning meaningful assert statements for unit test cases. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering (ICSE '20)*. ACM, New York, NY, 1398–1409. DOI : <https://doi.org/10.1145/3377811.3380429>
- [59] Yi Wu, Nan Jiang, Hung Viet Pham, Thibaud Lutellier, Jordan Davis, Lin Tan, Petr Babkin, and Sameena Shah. 2023. How effective are neural networks for fixing security vulnerabilities. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*. René Just and Gordon Fraser (Eds.), ACM, New York, NY, 1282–1294. DOI : <https://doi.org/10.1145/3597926.3598135>
- [60] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated program repair in the era of large pre-trained language models. In *Proceedings of the 45th International Conference on Software Engineering (ICSE '23)*. ACM, New York, NY.
- [61] Zhuokui Xie, Yinghao Chen, Chen Zhi, Shuiguang Deng, and Jianwei Yin. 2023. ChatUniTest: A ChatGPT-based automated unit test generation tool. arXiv:2305.04764. Retrieved from <https://arxiv.org/abs/2305.04764>
- [62] Hao Yu, Yiling Lou, Ke Sun, Dezhi Ran, Tao Xie, Dan Hao, Ying Li, Ge Li, and Qianxiang Wang. 2022. Automated assertion generation via information retrieval and its integration with deep learning. In *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*. ACM, New York, NY, 163–174. DOI : <https://doi.org/10.1145/3510003.3510149>
- [63] Wei Yuan, Quanjun Zhang, Tieke He, Chunrong Fang, Nguyen Quoc Viet Hung, Xiaodong Hao, and Hongzhi Yin. 2022. CIRCLE: Continual repair across programming languages. In *Proceedings of the 31st ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '22)*. Sukyoung Ryu and Yannis Smaragdakis (Eds.), ACM, New York, NY, 678–690. DOI : <https://doi.org/10.1145/3533767.3534219>
- [64] Zhiqiang Yuan, Yiling Lou, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, and Xin Peng. 2023. No more manual tests? Evaluating and improving ChatGPT for unit test generation. arXiv:2305.04207. Retrieved from <https://arxiv.org/abs/2305.04207>
- [65] Juan Zhai, Yu Shi, Minxue Pan, Guian Zhou, Yongxiang Liu, Chunrong Fang, Shiqing Ma, Lin Tan, and Xiangyu Zhang. 2020. C2S: Translating natural language comments to formal program specifications. In *Proceedings of the 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE '20)*. Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.), ACM, New York, NY, 25–37. DOI : <https://doi.org/10.1145/3368089.3409716>
- [66] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. 2024. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology* 33, 2 (2024), 55:1–55:69. DOI : <https://doi.org/10.1145/3631974>
- [67] Quanjun Zhang, Chunrong Fang, Tongke Zhang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. Gamma: Revisiting template-based automated program repair via mask prediction. In *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering (ASE '23)*. IEEE, 535–547. DOI : <https://doi.org/10.1109/ASE56229.2023.00063>
- [68] Quanjun Zhang and Bowen Yu. 2023. Replication Package. Retrieved September 28, 2023 from <https://github.com/iSEngLab/LLM4AG>
- [69] Quanjun Zhang, Tongke Zhang, Juan Zhai, Chunrong Fang, Bowen Yu, Weisong Sun, and Zhenyu Chen. 2023. A critical review of large language model on software engineering: An example from ChatGPT and automated program repair. arXiv:2310.08879. Retrieved from <https://arxiv.org/abs/2310.08879>

Received 16 December 2023; revised 2 July 2024; accepted 26 September 2024